

1. Datos generales

Curso	Inteligencia Artificial Aplicada al Control (ICA636-0001)
Trabajo	Tarea 3 — Modelamiento de Sistemas Dinámicos
Alumno	Joel Arzapalo Guerra
Código	20032006
Fecha	8 de julio de 2026

2. Introducción

Este trabajo analiza un conjunto de programas MATLAB que entrenan redes neuronales para **identificar (modelar) sistemas dinámicos** a partir de datos de entrada-salida. El código se organiza en dos familias:

- **Modelo estático** (`MotorNeuroEstatico.m`, `MotorNeuroEstatico1.m`): una red *sin realimentación* que aproxima la relación entrada-salida usando como entradas la excitación actual y valores *pasados* de la salida medida (con entradas y salidas retardadas). Se entrena con *retropropagación* estándar sobre un conjunto fijo de patrones.
- **Modelo dinámico** (`DynamicBPModelamiento2v.m`, `...3v.m`, `...DosIntermedias.m`, `DynamicBPLinealmodel12.m`): una red *recurrente* cuya salida se *realimenta* como entrada del siguiente paso. Se entrena con *retropropagación dinámica* (DBP), propagando las derivadas totales del costo a través del tiempo mediante la matriz Jacobiana de la red.

El informe se divide en dos partes: (i) el **análisis del código** de cada script, y (ii) los **resultados experimentales**, donde se entrenan las redes y se estudia cómo influyen los principales parámetros (ruido de medición, número de neuronas, tasa de aprendizaje, número de regresores, medición parcial, riqueza de la señal de entrada y arquitectura) en la calidad del modelo obtenido.

Para poder ejecutar los experimentos de forma reproducible, los scripts originales (interactivos, con `input()` y `load` de pesos previos) se adaptaron a versiones no interactivas: se eliminaron las llamadas a `input()` y `load`, se fijó la semilla del generador aleatorio (`rng`) y las rutinas de entrenamiento se encapsularon en funciones parametrizables (`lib_static.m`, `lib_dbp.m`) que conservan exactamente la misma matemática de los originales.

3. Análisis del código

3.1. Estructura común

Todos los scripts comparten el mismo esqueleto: (1) generación de la señal de entrada u_k ; (2) simulación de la planta para producir los datos de salida y_k^* ; (3) definición de la arquitectura de la red (ne entradas, nm neuronas ocultas, ns salidas) e inicialización de los pesos v , w y de los parámetros de la sigmoide (centro c , pendiente a); (4) el bucle de entrenamiento por descenso de gradiente; y (5) la visualización del ajuste y de la función de costo.

La activación oculta es una **sigmoide bipolar**

$$n = \frac{2}{1 + e^{-(m-c)/a}} - 1, \quad m = v^\top \text{in_red},$$

cuya derivada $\partial n / \partial m = (1 - n^2) / (2a)$ aparece en todas las reglas de la cadena. La salida de la red es lineal, $\text{out} = w^\top n$.

3.2. Modelo estático: MotorNeuroEstatico.m

La planta es un **motor** descrito por un sistema lineal de 3 estados (posición, velocidad, corriente) que se discretiza con `c2d` y se simula bajo una excitación de voltaje $v(k)$ saturada a ± 24 V. Para dar “dinámica artificial” a una red estática, se construyen regresores con la salida retardada:

```
1 x1 = volt;
2 x2(2:nv,1) = pos(1:nv-1,1); % pos(k-1)
3 x3(2:nv,1) = x2(1:nv-1,1); % pos(k-2)
4 % ... hasta x7 = pos(k-6)
5 xb = [ x1 x2 x3 x4 x5 x6 x7 ]; yb = pos;
```

Listing 1: Regresores (valores pasados): voltaje actual + 6 posiciones pasadas (`ne=7`).

Las entradas y salidas se **escalan** dividiendo por su valor absoluto máximo (`factx`, `facty`) para mantener las señales en $[-1, 1]$ y evitar la saturación de la sigmoide. El entrenamiento es por lotes: se acumula el gradiente sobre todos los patrones y se actualiza una vez por época.

```
1 for k = 1:nx
2     in = (xesc(k,:))';
3     m = v'*in;
4     n = 2.0./(1+exp(-m./a)) - 1;
5     out = w'*n;
6     er = out - (yesc(k,:))';
7     JJ = JJ + 0.5*er'*er;
8     dndm = (1 - n.*n)/2;
9     dJdw = dJdw + n*er';
10    dJdv = dJdv + in * (dndm.*(w*er))';
11 end
12 w = w - eta*dJdw/nx;
13 v = v - eta*dJdv/nx;
```

Listing 2: Núcleo del entrenamiento estático (retropropagación estándar).

La variante `MotorNeuroEstatico1.m` es idéntica pero usa solo $\{v_k, x_k, x_{k-1}, x_{k-2}\}$ (`ne = 4`): ilustra que el número de regresores m, n es un parámetro de diseño (relacionado con el orden del modelo lineal aproximado). Rasgo clave: al ser estática, la red **siempre necesita la salida medida como entrada**, también en operación, por lo que hereda el ruido de esa medición (se verifica en el experimento 4.3).

3.3. Modelo dinámico DBP: DynamicBPModelamiento2v.m

La planta es no lineal con 1 entrada y 2 estados, con un término *bilineal* z_1u :

$$z_1(k+1) = 0,3 z_1 - 0,4 z_2, \quad z_2(k+1) = 0,4 z_2 + 0,1 z_1 u + 0,5 u.$$

A diferencia del caso estático, la red corre en **lazo cerrado**: su salida se convierte en el estado de entrada del paso siguiente (`x = out_red`). Por eso el gradiente debe propagarse *a través del tiempo*. El script mantiene acumuladores recursivos de las *derivadas totales* $\partial y_i / \partial p$ que se actualizan con el Jacobiano de la red `jacob = w'*dndm*v(1:ne-1,:))'`:

```
1 jacob = w'*dndm*(v(1:ne-1,:))'; % Jacobiano 2x2 de la red
2 dy1dw_t = dy1dw_s + jacob(1,1).*dy1dw_t + jacob(1,2).*dy2dw_t;
3 dy2dw_t = dy2dw_s + jacob(2,1).*dy1dw_t + jacob(2,2).*dy2dw_t;
4 % (idem para v, c, a)
5 x = out_red; % la salida se realimenta como entrada
```

Listing 3: Recursión de derivadas totales (DBP) para 2 salidas.

El costo se pondera con los pesos q_1, q_2 , que implementan la **matriz de medición** Q : con $q = [1; 1]$ se miden ambos estados y con $q = [1; 0]$ solo el primero (medición parcial). El criterio de parada usa el error relativo $\sqrt{\sum e^2 / \sum y^{*2}}$.

Observación sobre la recursión. En el código original, la segunda línea (`dy2dw_t`) consume el `dy1dw_t` ya actualizado en el mismo paso, en lugar del valor del paso anterior. La forma matemáticamente correcta actualiza ambas sensibilidades de manera *simultánea* a partir de los valores previos; en la rutina de identificación lineal (Sec. 4.4) se usó esta forma correcta, lo que resulta determinante para que los parámetros converjan a su valor verdadero.

3.4. Sistema de 3 variables y red de dos capas

`DynamicBPModelamiento3v.m` extiende el DBP a un sistema de 3 salidas: el Jacobiano pasa a ser 3×3 y hay un acumulador de sensibilidad por cada salida. En el estado en que se entrega, este script usa **neuronas lineales** ($n = m$), por lo que la red equivale a un modelo de estado lineal aprendido.

`DynamicBPModelamientoDosIntermedias.m` usa **dos capas ocultas** (ur, v, w): la regla de la cadena atraviesa dos sigmoides (`dndm` y `dqdp`) y el Jacobiano acumula ambos factores (`jacob = w'*dqdp*v'*dndm*ur(1:ne-1,:)'`).

3.5. Identificación de sistema lineal: `DynamicBPLinealmodel2.m`

Caso especial y elegante: si el sistema es lineal $y_{k+1} = Ay_k + Bu_k$ y se usa una **red lineal de una sola capa** con $w = I$ fija, entonces la matriz de pesos v contiene directamente los coeficientes de A y B :

$$v = \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \\ b_1 & b_2 \end{bmatrix}.$$

Entrenar la red por DBP equivale entonces a **identificar los parámetros** del sistema. El script fija w (`w = w - 0*eta*dw`) y solo actualiza v .

4. Resultados experimentales

Todos los resultados se obtuvieron con las versiones no interactivas de los scripts, semilla `rng` fija para reproducibilidad. Las figuras se generan automáticamente con `Informe/exp/run_all.m`.

4.1. Modelo estático (motor)

Se entrenó la red estática del motor ($ne = 7$: voltaje + 6 posiciones pasadas, $nm = 20$, $\eta = 0,5$, 400 iteraciones). La Fig. 1 muestra la caída monótona del costo y la Fig. 2 el ajuste: la salida de la red (azul) reproduce la posición de la planta (rojo) con un error relativo de ajuste de 2,6 %.

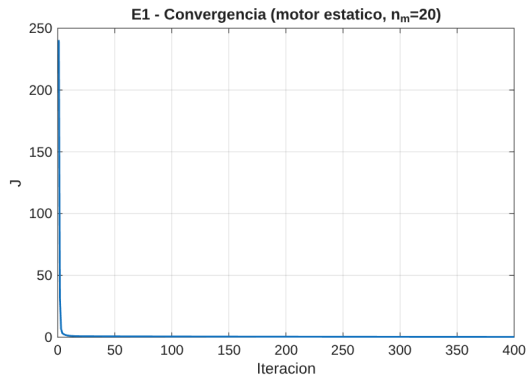


Figura 1: Convergencia del costo J (modelo estático, $n_m = 20$).

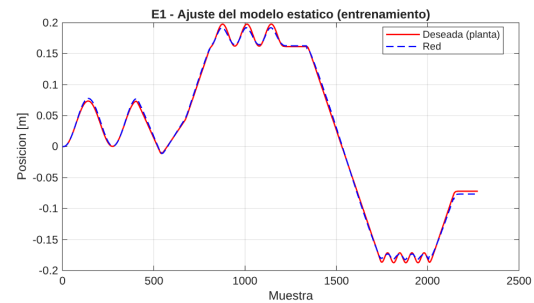


Figura 2: Ajuste del modelo estático sobre los datos de entrenamiento.

Influencia del ruido de medición. Como la red usa posiciones *medidas* pasadas como entradas, el ruido de medición degrada el modelo de forma directa. La Fig. 3 muestra que el RMSE (en unidades físicas, frente a la señal limpia) crece de $\approx 0,003$ m sin ruido a $\approx 0,038$ m con $\eta = 0,1$: un aumento de más de $10\times$. El ruido entra tanto por el objetivo como por los regresores, por lo que el modelo estático nunca se independiza de él.

Número de neuronas y de regresores. La Tabla 1 resume ambos barridos. Aumentar n_m reduce el error de ajuste (de 3,0 % con 3 neuronas a 1,3 % con 80), como cabe esperar por la mayor capacidad de la red (Fig. 4). En cambio, para esta planta casi lineal, agregar más posiciones pasadas *no* mejora el ajuste a iteraciones fijas: con un solo retardo el error es el menor (0,8 %) y crece al añadir regresores (Fig. 5), porque los parámetros adicionales ralentizan la convergencia sin aportar información dinámica nueva. La Fig. 6 confirma el compromiso habitual de la tasa de aprendizaje: η pequeño converge lento y η grande converge rápido pero con riesgo de oscilación.

Cuadro 1: Error relativo de ajuste del modelo estático.

Neuronas n_m	3	5	10	20	40	80
Error rel.	0.030	0.029	0.021	0.026	0.024	0.013
Regresores (# retardos)	1	2	3	6		
Error rel.	0.008	0.013	0.013	0.026		

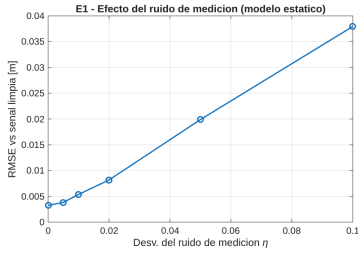


Figura 3: Efecto del ruido.

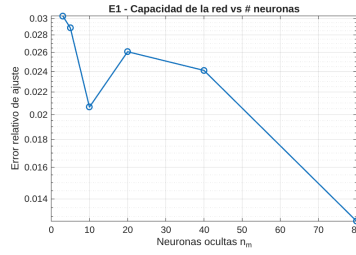


Figura 4: Error vs n_m .

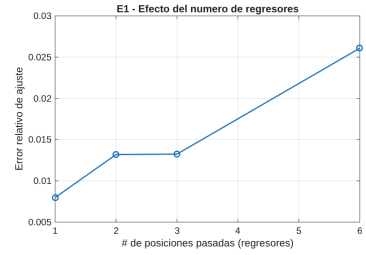


Figura 5: Error vs # regresores.

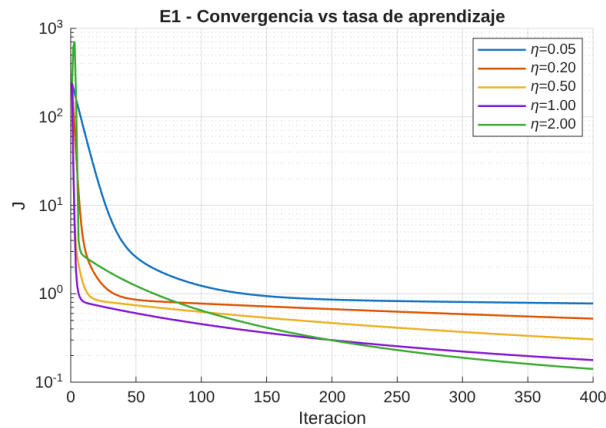


Figura 6: Convergencia del costo para distintas tasas de aprendizaje η .

4.2. Modelo dinámico DBP (2 estados)

Se entrenó la red recurrente sobre la planta no lineal ($n_m = 50$, $\eta = 0,05$). El error relativo total cae hasta $\approx 8,6\%$ (Fig. 7) y el modelo, ejecutado en **lazo cerrado** (simulación libre, sin

realimentar la señal real), sigue ambos estados z_1, z_2 de la planta (Fig. 8). Esto es notable: el modelo genera su propia trayectoria a partir solo de la entrada u .

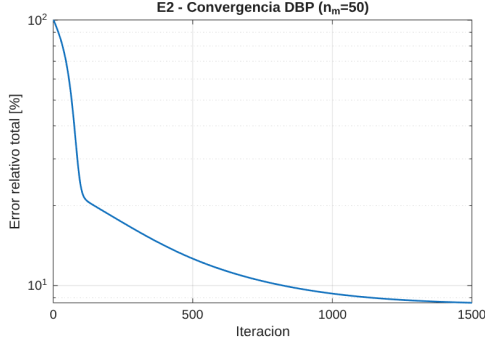


Figura 7: Convergencia del error relativo (DBP).

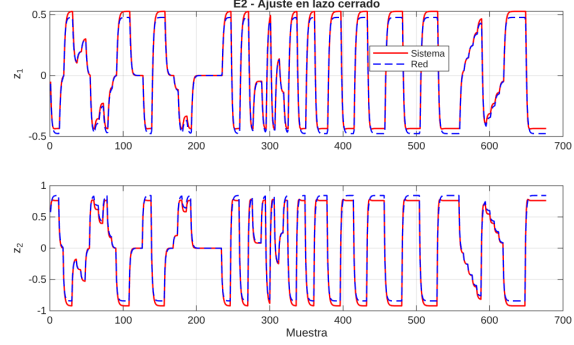


Figura 8: Ajuste en lazo cerrado de z_1 y z_2 .

Medición parcial (matriz Q). Con $q = [1; 1]$ se miden ambos estados y el RMSE en lazo cerrado es $[0,067, 0,060]$. Con $q = [1; 0]$ (solo se mide z_1), el estado no medido z_2 queda esencialmente sin observar y su RMSE se dispara a 0,68, arrastrando también a z_1 (0,16) —Fig. 9. Es decir: si un estado no se penaliza en el costo, el modelo no tiene incentivo para reproducirlo.

Neuronas y tipo de activación. El error final disminuye al aumentar n_m (Fig. 10). Por otro lado, como el término no lineal $0,1 z_1 u$ es débil frente a $0,5 u$, una red de **neuronas lineales** alcanza casi el mismo error que la sigmoideal (8,6 % en ambos casos, Fig. 11): la planta es “casi lineal” y no exige la capacidad no lineal de la sigmoide.

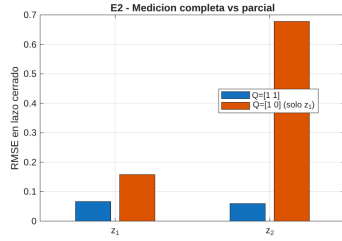


Figura 9: Medición completa vs parcial.

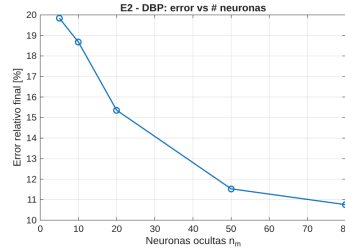


Figura 10: Error vs n_m .

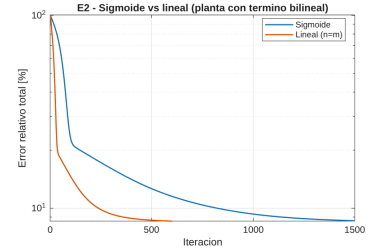


Figura 11: Sigmoide vs lineal.

4.3. Estático vs dinámico ante ruido de medición

Este es el experimento central. Sobre la *misma* planta no lineal se entrenan ambos modelos con datos de salida **contaminados por ruido** de desviación η , y se mide su error frente a la señal limpia. La diferencia clave está en el modo de operación: el modelo estático se evalúa en su uso nativo (predicción a un paso, alimentado con la salida *medida* ruidosa), mientras que el dinámico se evalúa en **lazo cerrado**, donde no vuelve a usar la medición.

Cuadro 2: RMSE del modelo frente a la señal limpia, según el ruido de medición η .

η	0	0.01	0.02	0.05	0.10	0.15
Estático (1 paso)	0.022	0.023	0.026	0.032	0.040	0.048
Dinámico (lazo cerrado)	0.053	0.053	0.053	0.053	0.054	0.054

La Fig. 12(a) muestra el error absoluto y (b) el error *normalizado* a su valor sin ruido. El resultado confirma la propiedad esperada: el error del modelo estático **crece** $\approx 2,15\times$ al pasar de $\eta = 0$ a $\eta = 0,15$, porque la medición ruidosa entra directamente por sus entradas y se propaga a la salida. El modelo dinámico, en cambio, permanece **prácticamente plano**: una vez entrenado, es independiente de la señal medida (genera su propia trayectoria), por lo que el ruido solo lo afectó durante el entrenamiento y allí se promedió. La Fig. 13 lo ilustra: con $\eta = 0,1$, la salida del modelo dinámico “filtra” el ruido y sigue la señal limpia, no la ruidosa.

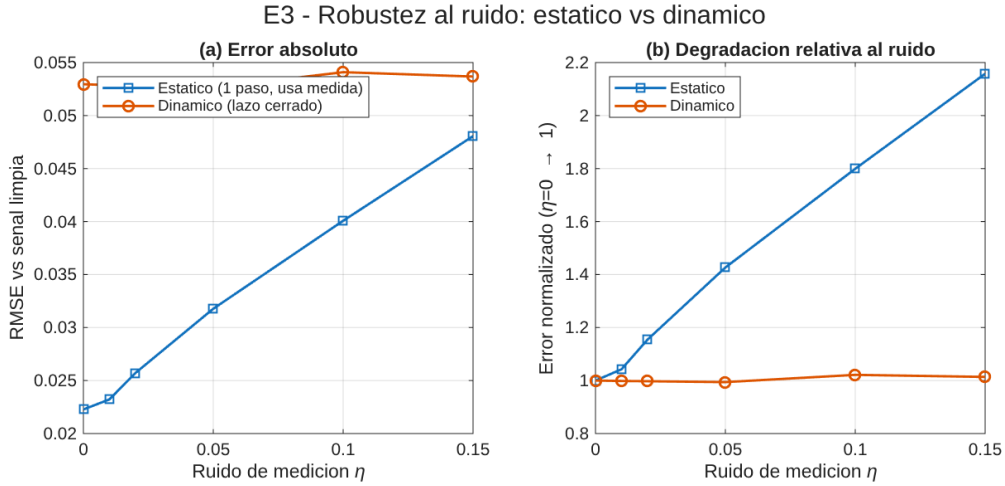


Figura 12: Robustez al ruido de medición. (a) RMSE absoluto; (b) degradación relativa (cada curva normalizada a su error con $\eta = 0$).

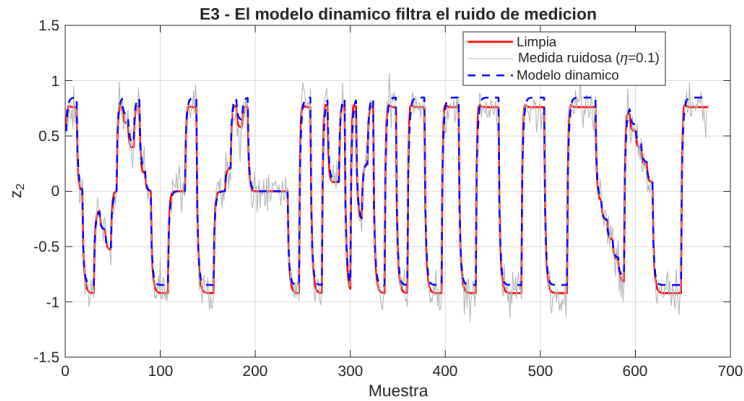


Figura 13: Con $\eta = 0,1$, el modelo dinámico reproduce la señal limpia y descarta el ruido de medición.

Nótese que, en términos absolutos, el modelo estático a un paso arranca con menor error (tarea más sencilla que la simulación libre); lo relevante aquí es la *tendencia*: el estático se degrada de forma monótona con el ruido y el dinámico no.

4.4. Identificación de parámetros de un sistema lineal

Se identificó el sistema lineal $y_{k+1} = Ay_k + Bu_k$ con $A = \begin{bmatrix} 0,6 & 0,8 \\ 0,3 & -0,1 \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ -0,2 \end{bmatrix}$, entrenando la red lineal de una capa (Sec. 3.5). Se compararon los dos modos de entrenamiento (Fig. 14): el de *lazo cerrado* del script original (error de salida) y el de *forzado con el estado medido* (error de ecuación, alimentando el estado medido).

Con forzado con el estado medido y entrada rica, la red **recupera exactamente** los parámetros (error $\|v - v_{\text{true}}\|_F \approx 2,5 \times 10^{-8}$):

$$v = \begin{bmatrix} 0,600 & 0,300 \\ 0,800 & -0,100 \\ 0,000 & -0,200 \end{bmatrix} = \begin{bmatrix} A^\top \\ B^\top \end{bmatrix} \text{ (exacto).}$$

En cambio, el modo de lazo cerrado del script queda en un error de 0,234: la optimización de error de salida es no convexa y más difícil, lo que explica por qué el script original recurre al *arranque desde pesos previos* (`load zz1v`).

Riqueza de la entrada. La identificabilidad depende críticamente de que la entrada excite todos los modos. Con la señal rica multinivel el error de parámetros es prácticamente nulo, mientras que con un escalón o una senoidal única sube a $\sim 5 \times 10^{-1}$ (Fig. 15): una entrada pobre no aporta información suficiente para determinar A y B .

Ruido de medición. El ruido introduce un **sesgo creciente** en los parámetros identificados (Fig. 16): al usar la salida medida (ruidosa) como regresor, el estimador de mínimos cuadrados se sesga (por el ruido presente en los regresores), y $\|v - v_{\text{true}}\|_F$ crece de forma monótona con η .

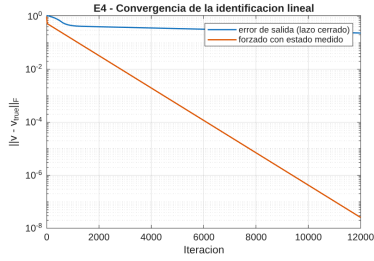


Figura 14: Convergencia: lazo cerrado vs forzado con el estado medido.

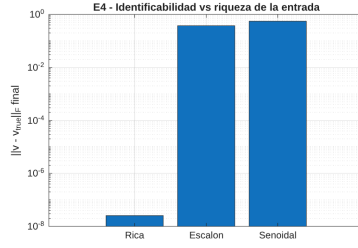


Figura 15: Error vs riqueza de la entrada.

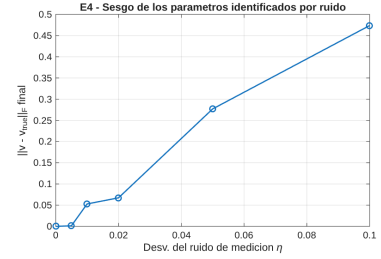


Figura 16: Sesgo por ruido de medición.

4.5. Arquitecturas: una vs dos capas y sistema de 3 variables

Una vs dos capas ocultas. Sobre la misma planta no lineal se comparó la red DBP de **una** capa oculta (12 neuronas) con la de **dos** capas (12–10). A igualdad de iteraciones, la red de una capa alcanzó un error final de 17,9% frente al 22,7% de la de dos capas (Fig. 17). Añadir una segunda capa *no* mejoró el resultado: aumenta el número de parámetros y la profundidad de la regla de la cadena (dos sigmoides en la recursión), lo que hace el entrenamiento más lento y con gradientes más difíciles de propagar. Para una planta de baja complejidad, una sola capa es suficiente y preferible.

Sistema de 3 variables. El DBP escala directamente a un sistema de 3 salidas (Jacobiano 3×3). El modelo lineal recurrente ajustó las tres variables con un error relativo de solo 1,7% (Fig. 18), confirmando que la metodología es general en el número de estados.

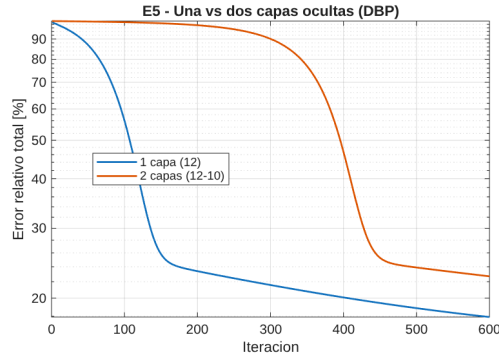


Figura 17: Convergencia: una vs dos capas ocultas.

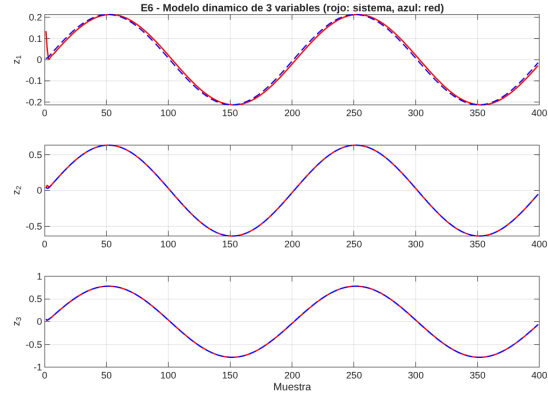


Figura 18: Modelo dinámico de un sistema de 3 variables.

5. Conclusiones

1. **Ambos enfoques modelan la dinámica**, pero por vías distintas: el modelo estático la reconstruye con regresores (entradas y salidas pasadas medidas) y *retropropagación* clásica; el dinámico la incorpora en la propia recurrencia y se entrena con DBP, propagando las derivadas totales a través del Jacobiano de la red.
2. **El ruido de medición separa claramente a las dos familias** (Sec. 4.3). El modelo estático usa la salida medida como entrada también en operación, por lo que su error crece de forma monótona con el ruido ($\approx 2,15\times$ entre $\eta = 0$ y $\eta = 0,15$). El modelo dinámico, tras el entrenamiento, es independiente de la señal medida y su error en lazo cerrado permanece esencialmente constante: filtra el ruido. Este es el argumento principal a favor del modelo dinámico cuando se lo usa como simulador.
3. **Los parámetros de diseño influyen de forma consistente**: más neuronas reducen el error de ajuste (E1, E2); una tasa de aprendizaje η mayor acelera la convergencia a costa de estabilidad (E1); y penalizar en el costo un estado que no se mide (matriz Q con un cero) hace que ese estado quede sin reproducir (E2).
4. **Complejidad adecuada al problema**: para plantas casi lineales, una red de neuronas lineales iguala a la sigmoideal (E2) y una sola capa oculta supera a dos capas (E5). Más capacidad no siempre es mejor a iteraciones fijas.
5. **La red lineal identifica los parámetros del sistema** (E4): sus pesos *son* las matrices A y B . La identificación es limpia cuando se dispone del estado medido (formulación de *error de ecuación*, convexa) y requiere una entrada **rica** para ser identificable; una señal pobre (escalón o senoidal única) no excita todos los modos y deja parámetros mal estimados. El ruido de medición introduce un sesgo creciente en los parámetros identificados. La variante en lazo cerrado del script original (error de salida) es más difícil de optimizar, lo que justifica el *arranque desde pesos previos* (`load zz1v`) que emplea.

A. Archivos del proyecto

Los experimentos se generan ejecutando `Informe/exp/run_all.m` en MATLAB. Las rutinas de entrenamiento reutilizables son `lib_static.m` (estático) y `lib_dbp.m` (dinámico, DBP vectorizado para *ns* salidas). Cada experimento E1–E6 tiene su propio script y guarda sus figuras en `Informe/figs/` y sus métricas en `*_metrics.mat`.